



COMP0497 – Algoritmos e Estrutura de Dados 1

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

- **COMP0497:** Algoritmos e Estruturas de Dados I
- 4 créditos – 60 horas
- **Horário:** 3N1234
- Utilizaremos o SIGAA para a comunicação e entregas

- Eficiência de algoritmos, notação assintótica
- Cálculo de complexidade de tempo e de espaço em algoritmos iterativos e recursivos
- Representação e manipulação de estruturas lineares de dados: listas, pilhas, filas
- Busca binária
- Hashing: funções, métodos e aplicações
- Árvores: binárias, binárias de busca, balanceadas
- Heaps e filas de prioridade
- Estrutura de dados para conjuntos disjuntos
- Complexidade das estruturas estudadas
- Algoritmos de ordenação por comparação
- Aplicações

Referências Básicas:

- SZWARCFITER, J. L.; MARKENZON, L. *Estruturas de Dados e Seus Algoritmos*. 3. ed. Rio de Janeiro: LTC, 2015.
- ZIVIANI, N. *Projeto de Algoritmos: Com Implementações em Pascal e C*. 3. ed. São Paulo: Cengage CTP, 2010.
- CELES, W.; CERQUEIRA, R.; RANGEL, J. L. *Introdução a Estruturas de Dados: Com Técnicas de Programação em C*. 2. ed. Rio de Janeiro: Elsevier, 2016.

Referências Complementares:

- PIVA JÚNIOR, D. et al. *Estruturas de Dados e Técnicas de Programação*. 1. ed. Rio de Janeiro: Campus, 2014.
- GOODRICH, M.; TAMASSIA, R. *Estruturas de Dados e Algoritmos em Java*. 4. ed. Porto Alegre: Bookman, 2007.

- SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms* (CLRS). 3. ed. MIT Press, 2009.

Recomendação: Livro online

Paulo Feofiloff (IME-USP): <https://www.ime.usp.br/~pf/algoritmos/>

A nota final é composta por duas unidades.

Composição da primeira unidade:

- Avaliação 1
- Exercícios valendo no máximo 5% da nota da primeira unidade

Composição da segunda unidade:

- Avaliação 2
- Exercícios valendo no máximo 5% da nota da primeira unidade

Repositiva

A prova de reposição é destinada aos alunos que faltaram a uma das avaliações. Ela terá o valor total de dez pontos.

Algoritmos e Análise de Algoritmos

Um **algoritmo** é qualquer procedimento computacional que recebe algum valor, ou um conjunto de valores, como *entrada* e produz algum valor, ou um conjunto de valores, como *saída*.

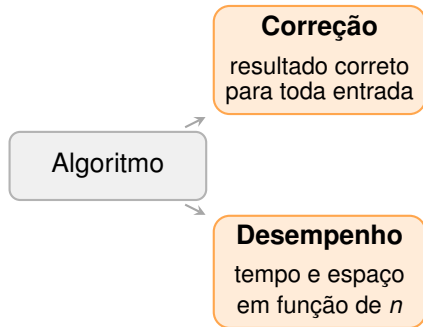
Um algoritmo é uma sequência de passos computacionais que transforma uma entrada em uma saída.

*“A análise de algoritmos é uma disciplina de **engenharia**. Um engenheiro civil, por exemplo, tem métodos e tecnologia para **prever o comportamento** de uma estrutura antes de construí-la.”*

“Da mesma forma, um projetista de algoritmos deve ser capaz de prever o comportamento de um algoritmo antes de implementá-lo.”

A análise de algoritmos avalia duas propriedades fundamentais de qualquer solução:

- **Correção:** o algoritmo produz o resultado esperado *para toda entrada válida*?
- **Desempenho:** qual o custo em *tempo* (operações) e *espaço* (memória) à medida que a entrada cresce?



Ambas devem ser satisfeitas — um algoritmo

Um algoritmo resolve **problemas computacionais** bem especificados.

Exemplo: Problema de Ordenação

- **Entrada:** Uma sequência de n números $a_1, a_2, \dots, a_n$.
- **Saída:** Uma permutação $a_1^*, a_2^*, \dots, a_n^*$ da entrada tal que $a_1^* \leq a_2^*, \dots, \leq a_n^*$.

A **instância** de um problema é um caso particular de uma entrada que satisfaz todas as restrições do problema.

Exemplo de instância: $\langle 42, 1, 2, 33, 52 \rangle$

Exemplo de instância para o problema de ordenação:

- Entrada: $\langle 42, 52, 1, 3, 5 \rangle$
- Saída: $\langle 1, 3, 5, 42, 52 \rangle$

- Um algoritmo é considerado **correto** se, para todas as instâncias, ele sempre termina com a solução para o problema.
- Quando um algoritmo é correto, dizemos que ele *resolve* o dado problema computacional.
- Um algoritmo incorreto pode não parar para algumas instâncias, ou pode parar com a resposta errada.
- **Observação:** um algoritmo incorreto ainda pode ser útil se conseguirmos controlar a taxa de erro do mesmo.

- Um algoritmo deve ser descrito da forma mais clara e precisa possível.
- É possível descrever um algoritmo utilizando a nossa própria língua nativa; às vezes, é a melhor forma de se fazer.
- Quando é necessário deixar tudo perfeitamente claro, utilizamos os **pseudocódigos**.

- Pseudocódigos são similares a C, C++, Java, Python, etc.
- Eles são desenvolvidos para um *humano* entender e não para a máquina entender.
- Aspectos da engenharia de software, como modularidade e gerenciamento de erros, são ignorados.
- O português é aceito ao escrever um pseudocódigo.

Linguagem Java

Vamos utilizar a **linguagem Java** para realizar a **implementação** dos algoritmos e estruturas de dados. Vamos utilizar pseudocódigo quando conveniente.

Pergunta

Qual o valor de S ao final da execução deste algoritmo? Há uma forma mais eficiente de se realizar a mesma conta?

Pseudocódigo

EXEMPLO-LACO-DUPLO(n)

```
1   $S = 0$ 
2  for  $i = 2$  to  $n - 2$ 
3      for  $j = i$  to  $n$ 
4           $S = S + 1$ 
5  return  $S$ 
```

Java

```
1  public class ExemploLacoDuplo
2  {
3      static int exemploLacoDuplo(int n) {
4          int S = 0;
5          for (int i = 2; i <= n - 2; i++) {
6              for (int j = i; j <= n; j++) {
7                  S = S + 1;
8              }
9          }
10         return S;
11     }
```

- **O Contexto:** Conta a lenda que, aos 7 anos, Carl Friedrich Gauss e sua turma receberam uma tarefa para mantê-los ocupados.
- **A Tarefa:** Somar todos os números inteiros de 1 a 100.
- **O Resultado:** Enquanto os colegas somavam um a um, Gauss entregou a resposta em poucos segundos: **5.050**.

Gauss percebeu que somar a sequência na ordem direta e inversa revelava um padrão:

$$\begin{array}{rcccccc} S = & 1 & + & 2 & + \dots + & 100 \\ S = & 100 & + & 99 & + \dots + & 1 \\ \hline 2S = & 101 & + & 101 & + \dots + & 101 & (\times 100) \end{array}$$

Portanto:

$$2S = 100 \times 101 \implies S = \frac{100 \times 101}{2} = 5.050$$

Para uma sequência de 1 até n , a soma S_n é dada por:

Fórmula de Gauss

$$S_n = \frac{(a_1 + a_n) \cdot n}{2}$$

- a_1 : Primeiro termo da sequência.
- a_n : Último termo da sequência.
- n : Quantidade de termos.

Por que isso é importante em computação?

- **Abordagem Iterativa (Loop):**
 - Realiza n adições.
- **Abordagem Analítica (Gauss):**
 - Apenas 3 operações matemáticas, independentemente do tamanho de n .

EXEMPLO-LACO-DUPLO(n)

```
1  S = 0
2  for i = 2 to n - 2
3      for j = i to n
4          S = S + 1
5  return S
```

Quantas vezes $S = S + 1$ executa?

- Para cada i , o laço j executa $n - i + 1$ vezes.
- **Sequência de execuções:**
 - $i = 2 \implies n - 1$
 - $i = 3 \implies n - 2$
 - ...
 - $i = n - 2 \implies 3$

A Conexão com Gauss

A soma total de execuções é a soma dos termos: $(n - 1) + (n - 2) + \dots + 3$.
É uma PA decrescente onde:

$$a_1 = n - 1, \quad a_k = 3, \quad \text{Total de termos } k = (n - 2) - 2 + 1 = n - 3$$

$$\text{Soma} = \frac{(a_1 + a_k) \cdot k}{2} = \frac{(n - 1 + 3) \cdot (n - 3)}{2} = \frac{(n + 2)(n - 3)}{2}$$

Notação assintótica

- **Introdução por Paul Bachmann**

Paul Bachmann criou a notação O em 1894 para representar o crescimento de funções matemáticas na teoria dos números.

- **Formalização por Edmund Landau**

Edmund Landau popularizou e formalizou os símbolos O e o em 1909, conhecidos como símbolos de Landau.

- **Aplicação na análise de algoritmos**

Dr. Knuth formalizou e popularizou o uso das notações assintóticas (Notação O , Ω e Θ), que pertencem à família da notação de Bachmann-Landau, no contexto da análise de algoritmos e na Ciência da Computação moderna.

Donald E. Knuth

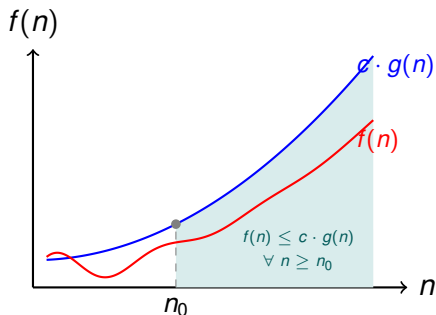
Considerado o pai da análise de algoritmos. Desenvolveu inúmeras técnicas e ajudou a estabelecer a área como disciplina formal.

Definição

Sejam $f(n)$ e $g(n)$ duas funções assintoticamente não-negativas.
Dizemos que $f \in O(g)$ se existem constantes $c > 0$ e $N > 0$ tais que:

$$f(n) \leq c \cdot g(n), \quad \forall n > N$$

- **Intuição:** Para todo n suficientemente grande, $f(n)$ é dominado por um múltiplo de $g(n)$.
- $g(n)$ atua como um **limite superior** (upper bound) para o crescimento de $f(n)$.



- **O Ponto n_0 :** Limiar a partir do qual $f(n) \leq c \cdot g(n)$ vale *permanentemente*.
- **A Constante c :** Escala $g(n)$ de modo que ela cubra $f(n)$ para todo $n \geq n_0$.

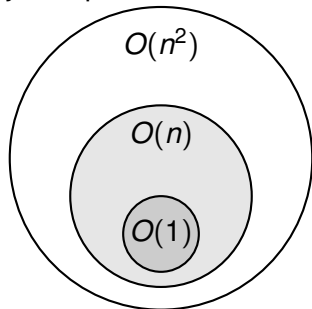
- Frequentemente escrevemos $f(n) = O(g(n))$, mas matematicamente isso é um **abuso de notação**.
- Na verdade, $O(g(n))$ é um **conjunto (classe)** de funções.
- A relação rigorosa é de pertinência: $f(n) \in O(g(n))$.

Definição Formal

$$O(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ tal que } 0 \leq f(n) \leq cg(n), \forall n \geq n_0\}$$

- $g(n)$ é um **limite superior assintótico** para todas as funções do conjunto.

- Como $O(g(n))$ é um conjunto, podemos observar relações de subconjuntos:



- Exemplo: $O(1) \subset O(n) \subset O(n^2)$.
- Dizer que uma função é constante implica que ela também é linear (em termos de limite superior).

$$10n + 50 \in O(n^2)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Pela definição: $f(n) \leq c \cdot g(n)$ para $n \geq n_0$.
- Queremos $10n + 50 \leq c \cdot n^2$.
- Se escolhermos $c = 1$: $10n + 50 \leq n^2 \implies n^2 - 10n - 50 \geq 0$.
- Isso é verdade para qualquer $n \geq 14$.
- **Conclusão:** Como existe $c = 1$ e $n_0 = 14$, a afirmação é verdadeira. $O(n^2)$ é um limite superior válido (embora não seja o mais "justo").

$$n^2 \in O(n)$$

Verdadeiro ou Falso?

Resposta: Falso.

- Suponha que existam c, n_0 tais que $n^2 \leq c \cdot n$.
- Dividindo por n (para $n > 0$): $n \leq c$.
- **Contradição:** c é uma constante fixa, mas n cresce ao infinito. Não existe uma constante c que seja sempre maior ou igual a n para todo $n \geq n_0$.
- Portanto, n^2 cresce estritamente mais rápido que n .

$$\log_{10}(n) \in O(\log_2(n))$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Lembre-se da mudança de base: $\log_{10}(n) = \frac{\log_2(n)}{\log_2(10)}$.
- Seja $c = \frac{1}{\log_2(10)} \approx 0.301$.
- Então $\log_{10}(n) = c \cdot \log_2(n)$.
- Como a diferença é apenas uma constante multiplicativa, as funções pertencem à mesma classe de complexidade.

$$3n^2 + 10n \in O(n^2)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Para grandes valores de n , o termo de maior grau domina.
- $3n^2 + 10n \leq 3n^2 + 10n^2 = 13n^2$ (para $n \geq 1$).
- Escolhendo $c = 13$ e $n_0 = 1$, a condição $f(n) \leq c \cdot g(n)$ é satisfeita.

$$2^n \in O(n^{100})$$

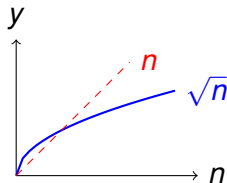
Verdadeiro ou Falso?

Resposta: Falso.

- Funções exponenciais sempre superam funções polinomiais assintoticamente.
- $\lim_{n \rightarrow \infty} \frac{2^n}{n^{100}} = \infty$.
- Não importa quão grande seja o expoente do polinômio, o crescimento de 2^n eventualmente ultrapassa qualquer $c \cdot n^{100}$.

$$\sqrt{n} \in O(n)$$

Verdadeiro ou Falso?



Resposta: Verdadeiro.

- Note que $\sqrt{n} \leq 1 \cdot n$ para todo $n \geq 1$.
- Como $n^{0.5}$ cresce mais devagar que n^1 , a relação de pertinência é válida.

$$n! \in O(2^n)$$

Verdadeiro ou Falso?

Resposta: Falso.

- Suponha, por contradição, que $n! \in O(2^n)$, ou seja, que existam $c > 0$ e n_0 tais que $n! \leq c \cdot 2^n$ para todo $n \geq n_0$.
- Observe que:

$$\frac{n!}{2^n} = \frac{1}{2} \cdot \frac{2}{2} \cdot \frac{3}{2} \cdots \frac{n}{2} \geq \frac{n}{2}$$

pois cada fator $\frac{k}{2} \geq \frac{1}{2} > 0$ e o último fator vale $\frac{n}{2}$.

- Logo $\lim_{n \rightarrow \infty} \frac{n!}{2^n} \geq \lim_{n \rightarrow \infty} \frac{n}{2} = \infty$, contradizendo $n! \leq c \cdot 2^n$.
- Portanto $n! \notin O(2^n)$.



$$\log(n^2) \in O(\log n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Pela propriedade dos logaritmos: $\log(n^2) = 2 \log n$.
- $2 \log n \leq c \cdot \log n$.
- Se escolhermos $c = 2$ e $n_0 = 1$, a igualdade/desigualdade se mantém.
- **Atenção:** Isso não funciona para $\log(n!)$, que é $O(n \log n)$.

$$2^n + n^{10} \in O(2^n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Em uma soma, o termo de maior crescimento domina a complexidade.
- Como $n^{10} \in O(2^n)$ (polinomial vs exponencial), existe um n_0 tal que $n^{10} \leq 2^n$.
- Logo, $2^n + n^{10} \leq 2^n + 2^n = 2 \cdot 2^n$.
- Escolhendo $c = 2$, a afirmação é verdadeira.

$$3^n \in O(2^n)$$

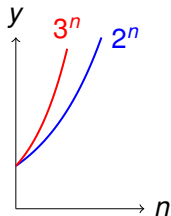
Verdadeiro ou Falso?

Resposta: Falso.

- Analisando a razão:

$$\lim_{n \rightarrow \infty} \frac{3^n}{2^n} = \lim_{n \rightarrow \infty} (1.5)^n = \infty$$

- Como a razão cresce ilimitadamente, não existe constante c que satisfaça $3^n \leq c \cdot 2^n$.



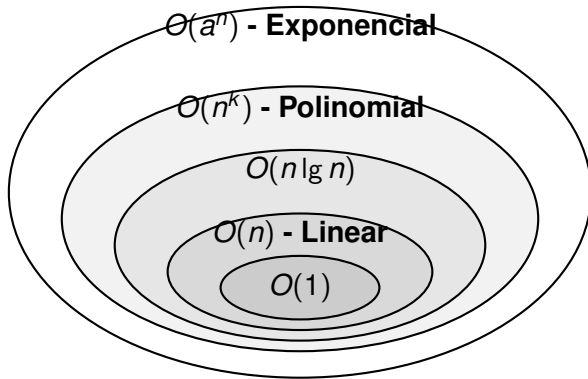
Regra de Ouro para Exponenciais

Diferente dos polinômios (onde $O(2n^2) = O(n^2)$), nas exponenciais a base **altera** a classe de complexidade:

$$a^n \in O(b^n) \iff a \leq b$$

Classe $O(g(n))$	Nome Comum
$O(1)$	constante
$O(\lg n)$	logarítmica
$O(n)$	linear
$O(n \lg n)$	$n \log n$
$O(n^2)$	quadrática
$O(n^3)$	cúbica
$O(n^k)$ com $k \geq 1$	polinomial
$O(2^n)$	exponencial
$O(a^n)$ com $a > 1$	exponencial

- Note que $O(1) \subset O(\lg n) \subset O(n) \cdots \subset O(a^n)$.
- Se uma função é linear, ela também é, por definição, quadrática.



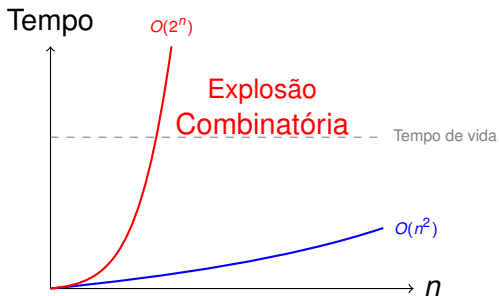
- Quanto mais "interno" o conjunto, mais eficiente é o algoritmo.
- Um algoritmo $O(n)$ "mora" dentro do conjunto das funções $O(n^2)$.

- A Notação O não é apenas teoria; ela define a **viabilidade** de um software.
- Considere um computador que realiza 10^9 operações por segundo:

n	$O(n \log n)$	$O(n^2)$	$O(2^n)$	$O(n!)$
10	< 1s	< 1s	< 1s	4s
50	< 1s	< 1s	36 anos	Inviável
100	< 1s	< 1s	10^{17} anos	Inviável
1.000	< 1s	1s	Inviável	Inviável
1.000.000	20s	12 dias	Inviável	Inviável

- Algoritmos polinomiais (n , n^2) lidam com milhões de dados.
- Algoritmos exponenciais morrem com apenas 50 ou 100 itens.

Onde o Problema "Explode"?



Conclusão Importante

Um algoritmo $O(2^n)$ pode até rodar rápido para $n = 10$, mas ele **não escala**. A escalabilidade é o coração da análise assintótica.

- Se $O(g(n))$ nos dá o "pior caso" ou teto, $\Omega(g(n))$ nos dá o "**melhor caso**" ou o piso de crescimento.
- Dizemos que $f(n) \in \Omega(g(n))$ se $f(n)$ cresce **pelo menos** tão rápido quanto $g(n)$.

Definição Formal

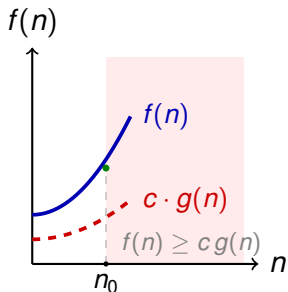
$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ tal que } 0 \leq cg(n) \leq f(n), \forall n \geq n_0\}$$

- Intuição: $g(n)$ é uma estimativa otimista. O algoritmo nunca será mais rápido (em termos de ordem) do que isso.

Definição formal:

$f(n) = \Omega(g(n))$ se $\exists c > 0, n_0 > 0$ tais que

$$f(n) \geq c \cdot g(n), \quad \forall n \geq n_0$$



Intuição:

- $g(n)$ é **cota inferior** de $f(n)$
- f cresce **pelo menos** tão rápido quanto g
- Região **sombreada**: onde a propriedade vale

- Existe uma relação direta e elegante entre as duas notações:

Afirmação	Equivalência
$f(n) \in O(g(n))$	$g(n) \in \Omega(f(n))$
$n \in O(n^2)$	$n^2 \in \Omega(n)$

- **Analogia com Números Reais:**
 - $f(n) \in O(g(n))$ funciona como $f \leq g$.
 - $f(n) \in \Omega(g(n))$ funciona como $f \geq g$.

$$n^2 \in \Omega(n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Queremos mostrar que $n^2 \geq c \cdot n$ para $n \geq n_0$.
- Dividindo por n (para $n > 0$): $n \geq c$.
- Se escolhermos $c = 1$, a inequação $n \geq 1$ é verdadeira para todo $n \geq 1$.
- Como n^2 cresce mais rápido que n , ele certamente tem n como um limite inferior.

$$n \in \Omega(n^2)$$

Verdadeiro ou Falso?

Resposta: Falso.

- Suposição: existam c, n_0 tais que $n \geq c \cdot n^2$.
- Dividindo por n : $1 \geq c \cdot n$, ou seja, $n \leq \frac{1}{c}$.
- **Contradição:** n cresce ao infinito, enquanto $1/c$ é uma constante. n não pode ser menor ou igual a uma constante para todo $n \geq n_0$.

$$5n^2 - 10n \in \Omega(n^2)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Queremos $5n^2 - 10n \geq c \cdot n^2$.
- Para $n \geq 10$, temos $10n \leq n^2$.
- Logo, $5n^2 - 10n \geq 5n^2 - n^2 = 4n^2$.
- Escolhendo $c = 4$ e $n_0 = 10$, a condição de limite inferior é satisfeita.

$$100 \in \Omega(1)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Queremos $100 \geq c \cdot 1$.
- Basta escolher $c = 100$ (ou qualquer valor ≤ 100) e $n_0 = 1$.
- Qualquer função constante é $\Omega(1)$ porque ela não decresce para zero.

$$\log n \in \Omega(\sqrt{n})$$

Verdadeiro ou Falso?

Afirmção: $\log n = \Omega(\sqrt{n})$

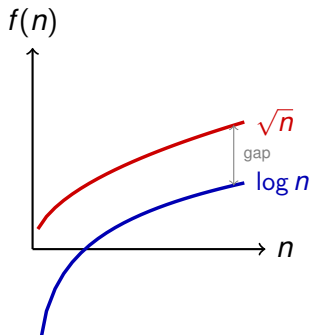
FALSO

Prova via limite:

$$\lim_{n \rightarrow \infty} \frac{\log n}{\sqrt{n}} = 0$$

Como o limite é 0, temos:

- $\log n \in O(\sqrt{n})$ ✓
- $\log n \notin \Omega(\sqrt{n})$ ✗
- $\log n \notin \Theta(\sqrt{n})$ ✗



$$2^n \in \Omega(n^{10})$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Exponenciais sempre dominam polinômios.
- $\lim_{n \rightarrow \infty} \frac{2^n}{n^{10}} = \infty$.
- Como a razão tende ao infinito, 2^n cresce muito mais rápido que n^{10} , satisfazendo a condição de limite inferior.

$$n \log n \in \Omega(n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Queremos $n \log n \geq c \cdot n$.
- Dividindo por n : $\log n \geq c$.
- Para qualquer c , existe um n_0 (ex: 2^c) tal que para todo $n \geq n_0$, $\log n$ é maior que c .
- Logo, $n \log n$ cresce ligeiramente mais rápido que n .

$$2^n \in \Omega(3^n)$$

Verdadeiro ou Falso?

Resposta: Falso.

- Analisando a razão: $\frac{2^n}{3^n} = (2/3)^n$.
- $\lim_{n \rightarrow \infty} (0.66)^n = 0$.
- Como a função 2^n se torna insignificante perto de 3^n conforme n cresce, ela não pode ser um limite inferior para 3^n .

$$n! \in \Omega(2^n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- $n!$ cresce mais rápido que qualquer função exponencial de base constante.
- $\frac{n!}{2^n} = \frac{1 \cdot 2 \cdot 3 \cdots n}{2 \cdot 2 \cdot 2 \cdots 2}$.
- Para $n > 4$, cada novo termo do fatorial é maior que 2, fazendo a razão crescer ao infinito.

$$\sqrt{n} \in \Omega(\log n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Embora ambas cresçam devagar, a raiz quadrada (polinomial) cresce mais rápido que o logaritmo.
- $\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{\log n} = \infty$ (pode ser provado por L'Hôpital).
- Portanto, $\sqrt{n}$ é um limite inferior para $\log n$.

- A notação $\Theta(g(n))$ combina o limite superior (O) e o inferior (Ω).

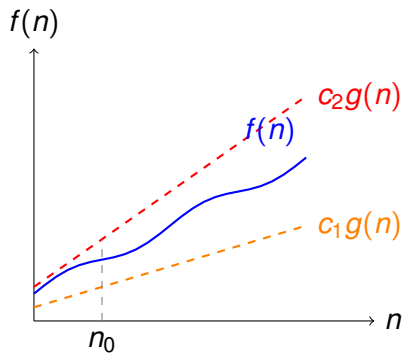
Definição Formal

$$f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \text{ e } f(n) \in \Omega(g(n))$$

- Ou seja, existem $c_1, c_2, n_0 > 0$ tais que:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \quad \forall n \geq n_0$$

- Isso significa que $f(n)$ cresce **exatamente** na mesma ordem que $g(n)$.



- Exemplo: $3n^2 + 5n + 1 \in \Theta(n^2)$.
- Ignoramos constantes e termos de menor ordem para encontrar o Θ .

Notação	Tipo de Limite	Analogia
$f(n) \in O(g(n))$	Superior (Teto)	$f \leq g$
$f(n) \in \Omega(g(n))$	Inferior (Piso)	$f \geq g$
$f(n) \in \Theta(g(n))$	Justo (Exato)	$f = g$

Propriedade Importante

Para provar que $f(n) \in \Theta(g(n))$, a estratégia mais comum é provar as duas inclusões separadamente (O e Ω).

$$\frac{1}{2}n^2 - 3n \in \Theta(n^2)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Para ser $\Theta(n^2)$, precisamos que seja $O(n^2)$ e $\Omega(n^2)$.
- **Limite Superior (O):** $\frac{1}{2}n^2 - 3n \leq \frac{1}{2}n^2$ para todo $n \geq 0$. Escolha $c_2 = 1/2$.
- **Limite Inferior (Ω):** Queremos $\frac{1}{2}n^2 - 3n \geq c_1 n^2$. Para $n \geq 12$, temos $3n \leq \frac{1}{4}n^2$. Logo, $\frac{1}{2}n^2 - 3n \geq \frac{1}{2}n^2 - \frac{1}{4}n^2 = \frac{1}{4}n^2$. Escolha $c_1 = 1/4$.
- Como a função está entre $\frac{1}{4}n^2$ e $\frac{1}{2}n^2$, ela é $\Theta(n^2)$.

$$n \in \Theta(n^2)$$

Verdadeiro ou Falso?

Resposta: Falso.

- Para ser $\Theta(n^2)$, a função deve ser $O(n^2)$ **E** $\Omega(n^2)$.
- $n \in O(n^2)$ é verdade (limite superior).
- No entanto, $n \notin \Omega(n^2)$ porque n cresce estritamente mais devagar que n^2 .
- Não existe constante $c_1 > 0$ tal que $n \geq c_1 n^2$ para todo n grande (pois $c_1 \leq 1/n$, e $1/n$ tende a zero).

$$\log_{10} n \in \Theta(\log_2 n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Pela mudança de base: $\log_{10} n = \frac{\log_2 n}{\log_2 10}$.
- Seja $K = \frac{1}{\log_2 10} \approx 0.301$.
- Então $f(n) = K \cdot \log_2 n$.
- Podemos escolher $c_1 = 0.3$ e $c_2 = 0.4$.
- $0.3 \log_2 n \leq 0.301 \log_2 n \leq 0.4 \log_2 n$.
- Constantes multiplicativas não alteram a classe Θ .

$$2^{n+1} \in \Theta(2^n)$$

Verdadeiro ou Falso?

Resposta: Verdadeiro.

- Note que $2^{n+1} = 2^1 \cdot 2^n = 2 \cdot 2^n$.
- Queremos $c_1 2^n \leq 2 \cdot 2^n \leq c_2 2^n$.
- Basta escolher $c_1 = 2$ e $c_2 = 2$ (ou qualquer intervalo que contenha o 2).
- Como a diferença é apenas uma constante multiplicativa fixa, a ordem de crescimento é a mesma.

$$4^n \in \Theta(2^n)$$

Verdadeiro ou Falso?

Resposta: Falso.

- Embora $2^n \in O(4^n)$ seja verdade, o contrário não é.
- Analisando a razão: $\frac{4^n}{2^n} = \left(\frac{4}{2}\right)^n = 2^n$.
- $\lim_{n \rightarrow \infty} 2^n = \infty$.
- Isso significa que 4^n cresce infinitamente mais rápido que 2^n .
- Portanto, $4^n \notin O(2^n)$, o que invalida a relação Θ .

Melhor, Pior e Caso Médio

O desempenho de um algoritmo depende não apenas do tamanho da entrada (n), mas também da **configuração específica** dos dados.

- **Pior Caso (*Worst Case*):** A entrada que maximiza o número de operações. Fornece uma **garantia superior**: nenhuma entrada fará o algoritmo ser mais lento.
- **Melhor Caso (*Best Case*):** A entrada que minimiza o número de operações. Útil para saber o mínimo de trabalho inevitável.
- **Caso Médio (*Average Case*):** A média do custo sobre todas as entradas possíveis, **ponderada por uma distribuição de probabilidade assumida**. Mais informativo que o pior caso, porém mais difícil de calcular.

Cuidado com um equívoco comum!

As notações O , Ω e Θ são **limitantes de funções**, não sinônimos de cenários. Podemos dizer, por exemplo, que o *pior caso* de um algoritmo é $\Theta(n)$ — usando qualquer notação em qualquer cenário.

Conceito	O que descreve
$O(g(n))$	Limitante superior de uma função ("cresce no máximo tão rápido quanto g ")
$\Omega(g(n))$	Limitante inferior de uma função ("cresce pelo menos tão rápido quanto g ")
$\Theta(g(n))$	Limitante justo ("cresce na mesma ordem que g ")
Pior caso	Um cenário — qual entrada é a mais custosa?
Melhor caso	Um cenário — qual entrada é a mais barata?
Caso médio	Um cenário — qual o custo esperado?

Combinação correta

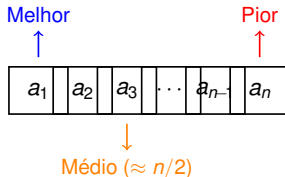
"O **pior caso** da busca linear é $\Theta(n)$ " — **cenário + limitante justo**.

"O **melhor caso** da busca linear é $\Theta(1)$ " — também válido.

Considere buscar um valor x em um vetor $A[1..n]$ não ordenado.

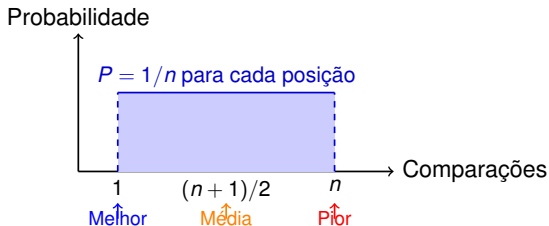
Análise de Cenários

- **Melhor Caso:** $x = A[1]$. $\Theta(1)$
- **Pior Caso:** $x = A[n]$ ou $x \notin A$. $\Theta(n)$
- **Caso Médio:** Supondo $x \in A$ e cada posição equiprovável, o valor esperado de comparações é $\frac{n+1}{2}$. $\Theta(n)$



Nota: Se x pode não estar em A , o caso médio depende da probabilidade de presença e altera o cálculo.

Para a busca linear (com posição equiprovável), a distribuição é **uniforme**, não gaussiana:

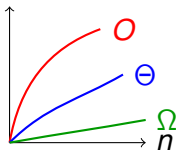


- Distribuições em sino (gaussianas) surgem quando o tempo depende da soma de **muitas variáveis aleatórias independentes** (Teorema Central do Limite) — não é o caso aqui.
- Na prática, priorizamos a análise de **pior caso** para oferecer garantias em sistemas críticos.

Conclusão

Vimos que a análise assintótica nos permite comparar algoritmos ignorando detalhes de hardware e focando no **crescimento** da função.

- $O(g(n))$: Limite Superior (Teto). "O algoritmo não é pior que isso."
- $\Omega(g(n))$: Limite Inferior (Piso). "O algoritmo não é melhor que isso."
- $\Theta(g(n))$: Limite Justo (Exato). "O algoritmo se comporta exatamente assim."



1. **Ignore as constantes:** $100n^2$ e $0.01n^2$ são ambos $\Theta(n^2)$ assintoticamente.
2. **Dominância:** O termo de maior grau sempre "vence" a longo prazo.
3. **Escalabilidade:** Algoritmos $O(2^n)$ ou $O(n!)$ são proibitivos para entradas grandes, não importa o quão rápido seja o seu processador.

Regrinha

$$f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \text{ e } f(n) \in \Omega(g(n))$$